

# WLCG standard system JWKS cache

*Authored by the WLCG AuthZ Working Group*

## Version History:

| Date       | Version | Comment                                           |
|------------|---------|---------------------------------------------------|
| 27.01.2026 | 1.0     | First version published on <a href="#">Zenodo</a> |

## Table of Contents

- [1. Introduction](#)
- [2. Motivating Examples](#)
- [3. JWKS Cache](#)
- [4. Storing a JWKS cache in a file](#)
- [5. Locating the cache file](#)
  - [5.1 Directory structure](#)
- [6. JWKS cache location algorithm](#)

## 1. Introduction

([ToC](#))

**Goal:** Provide a system-wide cache of JWKS information for issuers of interest that multiple token-parsing libraries can reuse. By having a cache in a trusted location on the system (and allowing the library to trust the system), we can reduce the impact of temporary outages of the JWKS endpoint, support clients that do not have direct access to the internet, reduce the load on the endpoint, and avoid rate-limiting conditions.

This document outlines the motivation behind the design choices and the design of the cache.

## 2. Motivating Examples

([ToC](#))

**Reliability of short-lived authorization services:** When a Pelican cache is embedded in a pilot, it will enforce WLCG token-based authorization but the lifetime of the service is the same as that of the pilot. If the service is started during unavailability of the issuer's JWKS endpoint, then having a copy of the JWKS on the system will allow the service to continue operation.

**Reliability of “lazy fetching” authorization libraries:** The scitokens-cpp library (and hence HTCondor- and XRootD-based services) will only fetch the JWKS on first use. If the first usage occurs during unavailability of the JWKS endpoint, then the access cannot be authorized. Having a system-level cache will keep a valid JWKS file at all times (assuming unavailability of the JWKS endpoint is less than the expiration of the JWKS copy).

**Avoiding rate limits on JWKS endpoints:** Some JWKS endpoints may have conservative rate limiting compared to the size of the computing resources. For example, if a large cluster is on an IPv6 /64 network, it may easily trigger a default rate limiter if worker nodes frequently trigger a JWKS cache query (such behavior has been observed in practice). A system-level cache enables administrators to manage the rate of outgoing queries to known endpoints.

### 3. JWKS Cache

([ToC](#))

The **JWKS** of an OIDC token issuer may be found by adding `/.well-known/openid-configuration` to the issuer URL and then reading the contents of the URL at the `jwks_uri` key found in the JSON object there.

Given the JWKS itself is a JSON object, we plan to have each entry (or file) in the cache serialized as a JSON object. The entry contains the **issuer name**, the **expiration time** of the entry, and the **JWKS** for the issuer.

An example serialized cache entry could have the following information as part of a JSON object (`https://demo.scitokens.org` is used for the issuer name):

```
{
  "https://demo.scitokens.org": {
    "expiration": 1755703888.4,
    "next_update": 1755700888,
    "jwks": {
      "keys": [
        {
          "alg": "ES256",
          "crv": "P-256",
          "kid": "b9ee",
          "kty": "EC",
          "use": "sig",
          "x": "ZsvEqnMaAWT5bMcIt5lx3HajCw6xfhl387U95JPoc0g=",
          "y": "D3-Bf0RKzeBSwoNdzkzcvalE7VTCM-gVTlWgSqPQm28="
        },
        {
          "alg": "RS256",
          ...
        }
      ]
    }
  }
}
```

The lookup key to the object is the issuer name (the contents of the `iss` claim in a JWT); we do not prescribe a specific serialization or normalization of the issuer URL for the cache entry. Key matching should be done by JSON UTF-8 string parsing rules.

The defined claims inside the entry's value are:

- `expiration`: A JSON numeric value. Interpreted as a Unix epoch time; after this time point, the contents of the cache entry should be ignored.
- `jwks`: The contents of the JWKS object for the issuer. The JWKS may be fetched from the `jwks_uri` key in the issuer's OIDC metadata discovery data.

Any undefined keys in the object (such as `next_update` in the above example) should be ignored by any parsing library as they may be used by the tool creating the cache.

A serialized JSON cache containing two entries for the issuers `https://demo.scitokens.org` and `https://cilogon.org` may have the following representation:

```
{
  "https://demo.scitokens.org": {
    "expiration": 1755703888,
    "next_update": 1755700888,
    "jwks": (JSON object of the JWKS)
  },
  "https://cilogon.org": {
    "expiration": 1755709999,
    "next_update": 1755709999,
    "jwks": (JSON object of the JWKS)
  }
}
```

## 4. Storing a JWKS cache in a file

[\(ToC\)](#)

A JWKS cache may be stored in a file using JSON's ASCII serialization. A parser should ignore any whitespace character bytes prior to the first { in the JSON object and after the last } in the JSON object. If any non-whitespace characters are encountered outside JSON object parsing, the file parsing must stop and any further caches listed must be ignored.

If there are multiple JSON objects in a file, they may be parsed as multiple caches and merged together into a single JSON object (subsequent keys should update prior keys; there is no mechanism for deleting a cache entry). This format is designed to allow system administrators to combine cache files together using simple Unix tools such as cat; JSON can otherwise be a little more difficult to concatenate (although jq -s add merges them pretty well).

An example of multiple JWKS caches in a single file is below:

```
{
  "https://demo.scitokens.org": {
    "expiration": 1755703888,
    "next_update": 1755700888,
    "jwks": (JSON object of the JWKS)
  }
}
{
  "https://cilogon.org": {
    "expiration": 1755709999,
    "next_update": 1755709999,
    "jwks": (JSON object of the JWKS)
  }
}
```

## 5. Locating the cache file

[\(ToC\)](#)

The system's JWKS cache may be located in a single file or in a directory. If a directory is specified, it should be searched for cache entry files as specified below. If at any point a cache entry for an issuer is found and its JWKS is not expired, further searches for additional caches for that issuer must stop.

## 5.1. Directory structure

[\(ToC\)](#)

If a directory is specified to be searched, the parser should search for a given issuer's cache using the following rules:

1. Create the SHA256 hash of the issuer URL and serialize the hash into hexadecimal ASCII. Use that as the file name.
2. Inside the directory, parse the file as a cache, making sure that there is an entry for the given issuer. If the file is not found or the given issuer is not in the file, return a code indicating that there was a cache miss, but if the file is found and has a parsing failure, return a code indicating that there was a parse error.

For example, for the issuer `https://demo.scitokens.org` and the specified directory named `Trusted-issuer-dir`, the JWKS may be found in the file

```
Trusted-issuer-dir /  
defd42fc2ae81f9628744cf6232e40c18697e65b5ef7e828c6223aa1b706bebb
```

## 6. JWKS cache location algorithm

[\(ToC\)](#)

The following locations should be searched in this order for an issuer's JWKS cache, stopping at the first match:

1. The directory indicated by the `JWKS_CACHE_DIR` environment variable.
2. A file indicated by the `JWKS_CACHE_FILE` environment variable.
3. A directory `/etc/jwks` for system administrator overrides.
4. A file `/etc/jwks/cache.json` for system administrator overrides.
5. The default cache directory location `/var/cache/jwks` (typically populated by an automated tool).
6. The default cache file location `/var/cache/jwks/cache.json`.

Here, a "match" is if the file exists *and* the desired issuer's JWKS has been cached *and* is valid.

Note - as an alternative, we considered having a `JWKS_CACHE_LOCATION` environment variable that's agnostic to whether it is a directory or a file; to avoid confusion for libraries, we decided it was more prudent to be explicit than implicit.